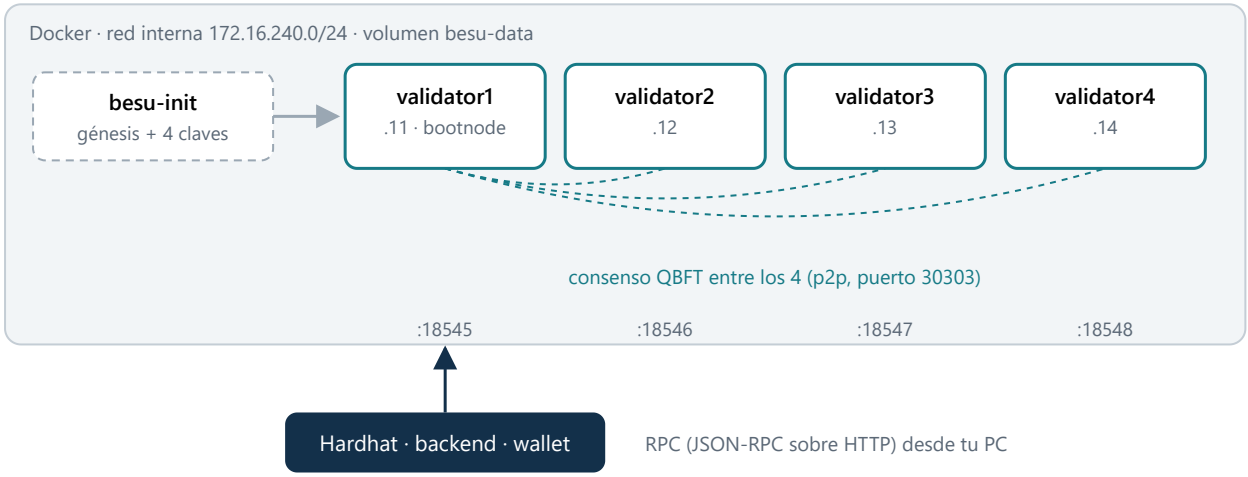
Red Ethereum local con Hyperledger Besu

FranToken · Resumen de lo implementado y por qué · 31 de octubre de 2026

> Se armó una red Ethereum privada de **4 nodos validadores** que corre en Docker, el equivalente a replicar los nodos de Hyperledger Fabric en local. Los contratos Solidity del proyecto se despliegan en ella sin cambios. Además quedó lista una variante de **un solo nodo** para Railway, donde se desplegará el backend.

1. Qué se construyó

Un contenedor inicializador genera una sola vez la configuración de la red (bloque génesis y claves). Después arrancan 4 validadores que se conectan entre sí y acuerdan cada bloque por consenso QBFT. Hardhat, el backend o una wallet se conectan a cualquiera de los nodos por RPC.



Por qué Besu y por qué QBFT

- **Besu** es el cliente Ethereum de la misma fundación que Fabric (Hyperledger / LF Decentralized Trust). Ejecuta los mismos contratos Solidity que la red pública, así que no hay que reescribir nada.
- **QBFT** es un consenso para redes privadas o de consorcio: un grupo conocido de validadores firma cada bloque. Con 4 validadores la red sigue funcionando si se cae 1 (regla 3f+1). Los bloques son finales en cuanto se producen y no se reorganizan, algo útil para un esquema permisionado como el de FranToken.
- Hardhat sigue siendo la herramienta de desarrollo y tests (es rápida). Besu agrega lo que Hardhat no simula: varios nodos, consenso, propagación entre pares y latencia real.

2. Paso a paso: archivos y decisiones

Paso 1 · Configuración de la red: `infra/besu/qbftConfigFile.json`

Parámetro	Valor y motivo
Chain ID	**2026**. Distinto del 31337 de Hardhat, para que una wallet o una firma de una red no se confunda con la otra.
Tiempo de bloque	**2 s**. Ethereum usa ~12 s; con 2 s la demo y el despliegue son más ágiles. Es un solo número si se quiere imitar la red pública.
Versiones del protocolo	Berlin, London y Shanghai desde el bloque 0: cubren lo que necesita el compilador del proyecto (Solidity 0.8.24, destino *paris*).
Cuentas precargadas	Las 6 primeras cuentas de prueba públicas de Hardhat, con 1.000 ETH de prueba cada una, para desplegar y simular las partes (franquiciante, fideicomiso, operadora, plataforma, verificador).
Validadores	4, generados automáticamente.

Paso 2 · Inicialización: `infra/besu/init-network.sh`

- Usa la herramienta oficial de Besu (`besu operator generate-blockchain-config`) para crear el génesis y las claves de los 4 validadores.
- Se ejecuta **una sola vez**: si la red ya existe, reutiliza génesis y claves y no hace nada.
- Las claves quedan **dentro del volumen de Docker**, nunca en el repositorio. Así lo exigen las reglas del proyecto sobre secretos.
- Calcula la dirección *enode* del nodo 1, que los otros nodos usan para encontrarlo.

Paso 3 · Arranque de cada nodo: `infra/besu/start-node.sh`

- Arranca Besu con su clave, el génesis y el RPC habilitado.
- Solo expone las APIs necesarias (`ETH, NET, WEB3, QBFT, TXPOOL`). Las administrativas quedan fuera.
- Los nodos 2 a 4 se conectan al nodo 1 como punto de encuentro (*bootnode*) y desde ahí descubren a los demás.

Paso 4 · Orquestación: `infra/besu/docker-compose.yml`

- **besu-init** corre primero. Los validadores esperan a que termine bien (`service_completed_successfully`).
- **IPs fijas** (172.16.240.11 a .14): la dirección enode de Besu necesita una IP.
- **Puertos 18545 a 18548** en tu PC, uno por nodo. No chocan con el nodo de Hardhat (8545), así que se pueden usar las dos cosas a la vez.
- **Memoria acotada** (`-Xmx768m`): Besu corre sobre Java y sin límite consume mucho.
- Imagen **fijada a la versión 26.9.0** (25 de septiembre de 2026), para que todos usen exactamente la misma.

Paso 5 · Integración con el proyecto

Archivo	Cambio y motivo
`hardhat.config.js`	Nueva red `besu` que apunta al nodo 1 y usa las cuentas de prueba públicas de Hardhat. La URL se puede cambiar con `BESU_RPC_URL`.
`scripts/deploy.js`	Acepta la red `besu` como red local. Hasta ahora solo aceptaba `hardhat` y `localhost`, porque usa montos de ejemplo.
`scripts/besu-status.js`	Muestra, para cada nodo, el bloque actual, su hash, los pares conectados y la cantidad de validadores. Sirve para comprobar el consenso.
`package.json`	Comandos `besu:up`, `besu:status`, `deploy:besu`, `besu:logs`, `besu:down` y `besu:reset`.
`.gitattributes`	Fuerza saltos de línea Linux (LF) en los `.sh`. En Windows, Git los convertiría a CRLF y los scripts fallarían dentro del contenedor.
`README.md`	Sección "Red local de varios nodos" con los comandos.

Paso 6 · Variante para Railway: `infra/besu/Dockerfile.railway`

- Un **único validador** en un solo contenedor, que se inicializa y arranca solo. Usa los mismos scripts que la red local.
- Necesita un **volumen en `/data`** para no perder la cadena ni la clave al reiniciar.
- El backend lo usa por la red privada de Railway: `http://<servicio>.railway.internal:8545`. No hace falta exponerlo a internet.
- Configuración en Railway: Root Directory `infra/besu` y variable `RAILWAY_DOCKERFILE_PATH=Dockerfile.railway`.

3. Cómo se usa

```
npm run besu:up        # primera vez: genera génesis y claves, levanta los 4 nodos
npm run besu:status    # bloque, hash, pares y validadores de cada nodo
npm run deploy:besu    # despliega los contratos de FranToken en la red
npm run besu:logs      # ver la actividad de los nodos
npm run besu:down      # detener (conserva la cadena)
npm run besu:reset     # borrar todo: bloques, génesis y claves
```

Requisito: Docker Desktop abierto. La primera vez descarga la imagen de Besu (~500 MB).

4. Verificación realizada

Todo se probó en esta PC (Docker Desktop 29.6.1, Windows 11) el 31 de octubre de 2026.

Prueba	Resultado
Levantar la red	**OK: génesis y 4 claves generados; los 4 validadores arriba.**
Consenso (`besu:status`)	**OK: los 4 nodos en el mismo bloque y con el mismo hash, 3 pares cada uno, 4 validadores, chain ID 2026.**

Despliegue de los contratos (`deploy:besu`)	**OK: se desplegaron los 8 contratos (token, registro, compliance, stablecoin de prueba y contratos C1 a C4).**
Replicación	**OK: el token desplegado a través del nodo 1 se lee igual desde el nodo 4 ("Cafe Central Polanco", tope 10.000).**
Tolerancia a fallas	**OK: con el validador 4 detenido, la red siguió produciendo bloques (del 174 al 183 en 30 s). Al reiniciarlo se resincronizó con el mismo hash que los demás.**
Variante Railway (imagen probada en local)	**OK: un nodo produce bloques. Tras reiniciarse conservó la cadena (siguió desde el bloque 28) y reutilizó génesis y clave.**
Tests de contratos (Hardhat)	**OK: 51 tests pasando, sin cambios en los contratos.**

Observación de rendimiento: en esta PC cada nodo usa ~600 MB de RAM y la CPU estuvo muy exigida. Las respuestas del RPC tardaron hasta ~7 s y el despliegue completo, ~5 minutos. Con menos carga es bastante más rápido. Si molesta, se puede bajar a 1 nodo o subir el tiempo de bloque.

5. Costos estimados en Railway

Precios publicados por Railway: plan Hobby USD 5 al mes con USD 5 de consumo incluido; RAM a USD 10 por GB al mes, CPU a USD 20 por vCPU al mes y disco a USD 0,15 por GB al mes. Las cifras de la tabla son **estimaciones**, no mediciones en Railway, y suponen ~1 GB por nodo, cerca de los ~600 MB medidos en local.

Opción	Dónde	Costo mensual estimado
4 validadores (este docker-compose)	Tu PC	USD 0
1 validador (`Dockerfile.railway`)	Railway	~USD 12–15
4 validadores	Railway	~USD 50–60
Sepolia (red de prueba pública) con un proveedor de RPC	Externo	USD 0 en planes gratuitos

Recomendación: 4 nodos en local para desarrollo y demostración, y 1 nodo en Railway (o Sepolia) para el backend desplegado. Para pasar de una red a otra solo cambia la URL del RPC.

6. Límites y pendientes

- **Es un entorno de prueba.** No reemplaza la decisión de red de producción (D10 en `docs/DECISIONS.md`, todavía pendiente).
- Las cuentas que despliegan son las de prueba **públicas** de Hardhat: cualquiera conoce sus claves. Sirven solo para esta red.
- El RPC acepta conexiones de cualquier origen (CORS abierto) para facilitar el desarrollo. Para un entorno compartido habría que restringirlo.
- Un solo validador en Railway no tolera fallas: si se cae, la red se detiene. Los 4 nodos de Railway que comparten un mismo proveedor tampoco forman una red realmente descentralizada.
- **La variante de Railway no se probó en Railway todavía.** Se verificaron los scripts y la red local, que comparten el mismo código.

- Railway puede importar un docker-compose, pero no toda su configuración (por ejemplo, un volumen compartido entre servicios). Por eso la variante para Railway es un Dockerfile de un solo servicio.

Fuentes: railway.com/pricing · docs.railway.com/guides/dockerfiles · docs.railway.com · private networking · github.com/hyperledger/besu/releases. Consultadas el 31/10/2026.